C# Ders - 23 Ekim 2026 Ders Notları

1. Dizi Mimarileri: Tek Boyutlu, Çok Boyutlu ve Düzensiz Diziler

C#'ta veriler karmaşıklaştıkça tek bir sıra halinde dizilmiş listeler yetersiz kalır. Bu noktada belleğin organize ediliş biçimine göre 3 farklı dizi mimarisi karşımıza çıkar:

```
1. Tek Boyutlu Dizi (1D Array):
   [ Elaman 0 | Eleman 1 | Eleman 2 | Eleman 3 ]
   (Tek satır, yatay liste)

2. Çok Boyutlu Matris Dizi (2D Rectangular Array):
   Satır 0 -> [ (0,0) | (0,1) | (0,2) ]
   Satır 1 -> [ (1,0) | (1,1) | (1,2) ]
   (Tüm satırların sütun sayısı EŞİTTİR, düzgün dikdörtgen/kare tablo)

3. Düzensiz Dizi (Jagged Array - Dizilerin Dizisi):
   Satır 0 -> [ 1 | 2 | 3 ]              (3 elemanlı)
   Satır 1 -> [ 1 | 2 | 3 | 8 | 9 ]      (5 elemanlı)
   Satır 2 -> [ 3 ]                      (1 elemanlı)
   (Her satırın boyutu birbirinden TAMAMEN BAĞIMSIZDIR)
```

A. Tek Boyutlu Diziler (1D Arrays)

Bellekte yan yana tek sıra halinde duran en basit yapıdır.

C#

```
int[] numbers = { 5, 6, 7, 8, 9 };
Console.WriteLine(numbers[0]); // İlk eleman
```

B. Çok Boyutlu / Matris Diziler (Multidimensional / Rectangular Arrays)

Excel tablosu veya bir apartman kat planı gibi **Satır ×× Sütun** eksenine sahip tablolardır. Virgül (`[,]`) işareti ile tanımlanırlar.

- **Kritik Kural:** Her satırın sütun sayısı **birbirine eşit** olmak zorundadır (Örn: 3 katlı bir binanın her katında kesinlikle 4 daire olması gibi).

Tanımlama ve Erişim:

C#

```
// 3 satır (kat), 4 sütun (daire) içeren 2 boyutlu metin dizisi:
string[,] apartman = new string[3, 4];

// Değer Atama: apartman[satirIndeksi, sutunIndeksi]
apartman[0, 0] = "Ahmet Bey (1. Kat 1. Daire)";
apartman[2, 3] = "Mehmet Bey (3. Kat 4. Daire)";
```

Çok Boyutlu Dizilerde Döngü Kurma (`GetLength` Mantığı):

Tek boyutlu dizilerdeki `.Length` tüm hücrelerin toplam sayısını (3×4=123×4=12) verir. Bu nedenle satır ve sütun sınırlarını ayrı ayrı almak için `.GetLength(boyutNumarasi)` kullanılır:

- `.GetLength(0)`: 1. boyutu, yani **Satır Sayısını** verir.
- `.GetLength(1)`: 2. boyutu, yani **Sütun Sayısını** verir.

C#

```
// İç içe döngü ile matrisi doldurma ve okuma
for (int i = 0; i < apartman.GetLength(0); i++) // Katlar (Satırlar)
{
    for (int j = 0; j < apartman.GetLength(1); j++) // Daireler (Sütunlar)
    {
        Console.WriteLine($"{i + 1}. Kat {j + 1}. Daire: {apartman[i, j]}");
    }
}
```

C. Düzensiz Diziler (Jagged Arrays - Dizilerin Dizisi)

Bellekte her satırı farklı uzunlukta olabilen esnek yapılardır. Çift köşeli parantez (`[][]`) ile tanımlanırlar.

- **Neden Kullanılır?** Örneğin bir okuldaki şubelerin öğrenci sayıları eşit değildir (A şubesi 20 kişi, B şubesi 15 kişi).Matris dizi kullanılırsa boş kalan yerler için gereksiz RAM israfı olur. Düzensiz dizilerde her satıra ihtiyacı kadar bellek açılır.

Tanımlama ve Bellek Tahsisi:

C#

```
// 1. Adım: Ana çatıyı oluştur (3 satırlı bir dizi)
int[][] numbers = new int[3][];

// 2. Adım: Her satırın kendi iç dizisini bağımsız olarak bellekte aç
numbers[0] = new int[] { 1, 2, 3 };            // 3 elemanlı
numbers[1] = new int[] { 1, 2, 3, 8, 9 };      // 5 elemanlı
numbers[2] = new int[] { 3 };                  // 1 elemanlı
```

Düzensiz Dizileri Ekrana Yazdırma:

Dış döngü satırları (`numbers.Length`), iç döngü ise o anki satırın kendi uzunluğunu (`numbers[i].Length`) gezer:

C#

```
for (int i = 0; i < numbers.Length; i++)
{
    Console.WriteLine($"--- {i}. Satır Grubu ---");
    for (int j = 0; j < numbers[i].Length; j++)
    {
        Console.Write(numbers[i][j] + " ");
    }
    Console.WriteLine();
}
```

2. Hazır `Array` Sınıfı Metotları

.NET kütüphanesinde dizileri sıralamak, aramak, kopyalamak ve boyutlandırmak için optimize edilmiş statik `Array` metotları bulunur.

Örnek başlangıç dizimiz:

C#

```
string[] names = { "Şukufe", "Arzu", "Elif", "Ahmet", "Abdullah", "Zübeyde" };
```

A. Sıralama ve Ters Çevirme

- `Array.Sort(dizi)`: Dizi elemanlarını küçükten büyüğe (alfabetik veya sayısal) sıralar. Doğrudan orijinal diziyi değiştirir.C#

  ```
  Array.Sort(names);
  // Sıralama: Abdullah, Ahmet, Arzu, Elif, Şukufe, Zübeyde
  ```

- `Array.Reverse(dizi)`: Dizinin mevcut eleman sırasını baştan sona tersine çevirir.

  C#

  ```
  Array.Reverse(names);
  // Sıralama: Zübeyde, Şukufe, Elif, Arzu, Ahmet, Abdullah
  ```

B. Arama ve İndeks Bulma

- `Array.IndexOf(dizi, arananDeger)`: Aranan elemanın dizideki **ilk indeks numarasını** döner. Eğer eleman dizide yoksa `-1` döner.C#

  ```
  int index = Array.IndexOf(names, "Şukufe"); // Bulursa indeksini, bulamazsa -1 döner
  ```

- `Array.Exists(dizi, kosul)`: Belirtilen koşulu sağlayan en az bir eleman var mı yok mu kontrol eder (`true` / `false`).C#

  ```
  bool varMi = Array.Exists(names, n => n == "Abdullah"); // true
  ```

- `Array.Find(dizi, kosul)`: Belirtilen koşula uyan **ilk elemanın değerini** döner.C#

  ```
  // 'A' harfi ile başlayan ilk ismi getir:
  string ilkA = Array.Find(names, n => n.StartsWith("A")); // "Abdullah"
  ```

  *(Not: `n => n.StartsWith("A")` yazımı bir **Lambda İfadesidir**. Dizideki her bir elemanı temsil eden sanal bir `n` değişkeni açıp "n, 'A' ile başlıyor mu?" şartını test eder).*

C. Kopyalama ve Boyut Değiştirme

- `Array.Copy(kaynak, kaynakBaslangic, hedef, hedefBaslangic, adet)`: Bir dizinin belirli bir aralığını başka bir diziye kopyalar.C#

```
string[] names2 = new string[10]; // 10 elemanlı boş dizi (içi null dolu)

// names dizisinin 3. indeksinden başla, 2 elemanı al, names2'nin 3. indeksinden itibaren yapıştır:
Array.Copy(names, 3, names2, 3, 2);
```

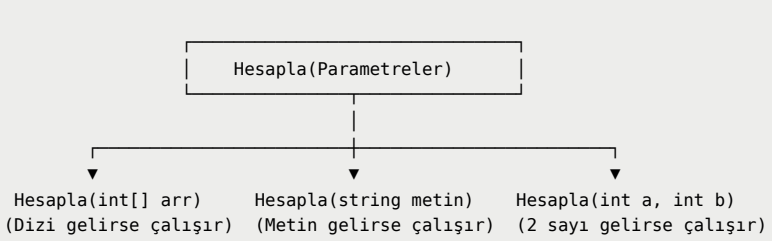
- `Array.Resize(ref dizi, yeniBoyut)`: Diziler bellekte sabit boyutlu üretilirler. Boyutlarını sonradan doğrudan değiştiremezsiniz. `Array.Resize` arka planda şu işlemi yapar:C#
  1. İstenen yeni boyutta tamamen yeni bir dizi açar.
  2. Eski dizideki elemanları bu yeni diziye kopyalar.
  3. `ref` sayesinde değişkeninizin adresini bu yeni diziye bağlar.

```
Array.Resize(ref names, 15); // names dizisi artık 15 elemanlıdır (artan yerler null/default olur)
```

3. Metot Aşırı Yükleme (Method Overloading) Nedir?

Bir sınıfta **aynı isimde birden fazla metot tanımlanmasına** "Metot Aşırı Yükleme (Method Overloading)" denir.

Gündelik hayattan düşünelim: "Açmak" fiilini hem kapı için kullanırız ("Kapıyı aç"), hem bilgisayar için ("Bilgisayarı aç"), hem de hesap için ("Banka hesabı aç"). Fiil aynıdır ama nesneye göre yapılan işlem farklılaşır. Yazılımda da aynı eylemi farklı parametrelerle yapmak istediğimizde `ToplaInt`, `ToplaDouble`, `ToplaUcSayi` gibi çirkin isimler üretmek yerine tek bir `Topla` ismi kullanırız.



Overloading (Aşırı Yükleme) Kuralları

Derleyici, hangi metodun çalışacağına **Method Signature (Metot İmzası)** üzerinden karar verir:

Metot İmzası=Metot Adı+Parametre Sayısı ve TipleriMetot İmzası=Metot Adı+Parametre Sayısı ve Tipleri

1. **İsimler Kesinlikle Aynı Olmalıdır:** Metotların isimleri birebir aynı kalır.
2. **Parametre Farklılığı Zorunludur:**
   - Ya parametre **tipleri** farklı olmalıdır (`int` vs `string`).

- Ya parametre **sayısı** farklı olmalıdır (2 parametre vs 3 parametre).
- Ya da parametrelerin **sıralaması** farklı olmalıdır (`(int, string)` vs `(string, int)`).

3. **Dönüş Tipi İmzaya Dahil Değildir!**

   İki metodun sadece dönüş tipi (`void` veya `int`) farklıysa derleyici bunu ayırt edemez ve **hata verir**. Parametrelerin kesinlikle farklılaşması gerekir.

Kod İncelemesi:

C#

```
// 1. Versiyon: Dizi alan metot
void Hesapla(int[] arr)
{
    Console.WriteLine("Dizi hesaplandı.");
}

// 2. Versiyon: Metin alan metot
int Hesapla(string metin)
{
    Console.WriteLine("Metin işlendi.");
    return metin.Length;
}

// 3. Versiyon: İki sayı alan metot
int Hesapla(int a, int b)
{
    return a + b;
}
```

Siz `Hesapla("merhaba")` dediğiniz anda derleyici parametrenin `string` olduğunu görür ve otomatik olarak 2. versiyonu çalıştırır.

4. Bütünleşik Örnek: Kodun Mantıksal Özeti (At Yarışı & Bahis Mekaniği)

Ders kodunun ilk yarısındaki At Yarışı simülasyonu, şu ana kadar öğrenilen tüm mekanizmaların nasıl bir araya geldiğini gösteren tam bir konsol oyun motorudur:

1. **Oyun Döngüsü (`while (islem && bakiye > 0)`):** Kullanıcının parası bitene veya çıkmak isteyene kadar oyun alanını açık tutar.
2. **Ekran Tazeleme ve Animasyon (`Console.Clear()` & `Thread.Sleep(100)`):** Yarıştaki her turda ekran temizlenip atların aldıkları mesafeler (tire işaretleri uzatılarak) yeniden çizilir. 100 milisaniyelik gecikme göze animasyon akıcılığı sağlar.
3. **Rastgele İlerleme Mantığı (`Random`):**
   - Kullanıcının seçtiği at: `rnd.Next(1, 3)` (Hileli mekanizma: Oyuncunun atı daha yavaş ilerletilir).
   - Diğer atlar: `rnd.Next(1, 5)` (Kasa avantajı).
4. **Modüler Metot Tasarımı:**
   - `StartRace(...)` : Yarışı başlatır ve kazanan atın indeksini döner.

- `WinControl(distance)`: Mesafe 50 birime ulaştı mı kontrolünü yapar (`bool`).
- `BahisKoy()`: Kullanıcıdan bahis miktarını doğrulatıp alır.
- `AtSec(...)`: At listesini yazdırıp kullanıcının tercihini alır.

5. Konu Özeti Tablosu

Yapı / Metot	Bellek / Davranış Biçimi	Temel Kullanım Amacı
`string[,]`	Matris (Dikdörtgen Tablo)	Satır ve sütun sayısı sabit, nizamî tablolar (Koltuk düzeni, bina planı).
`int[][]`	Düzensiz Dizi (Jagged)	Satır uzunlukları birbirinden farklı listeler grubu.
`Array.Sort()`	Yerinde Değişiklik	Diziyi alfabetik veya sayısal olarak küçükten büyüğe dizer.
`Array.Find()`	Lambda Eşleme	Koşulu sağlayan ilk elemanı yakalar.
`Array.Resize()`	Yeniden Bellek Tahsisi	Dizi boyutunu eleman kaybetmeden genişletir veya daraltır.
`Overloading`	Derleme Zamanı Polimorfizmi	Aynı isimli metoda farklı parametre kombinasyonları tanımlama yeteneği.